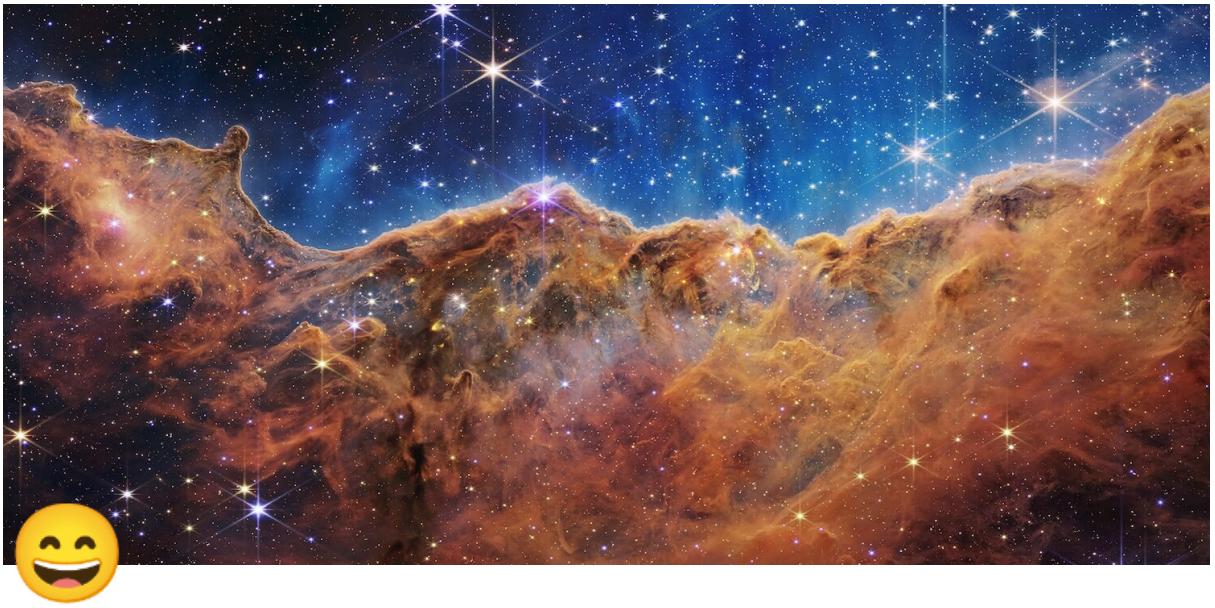

# Resi-Gig — Product Requirements Document (PRD)

🧭 **Purpose:** This PRD is the single source of truth for building the **Resi-Gig** MVP demo (AI-powered parametric income protection for India's gig riders) on Guidewire Cloud.
**MVP boundary:** No real money movement. No regulatory compliance handled in MVP. No native mobile app. No live integrations with actual gig platforms. English and Hindi only.


1) Executive Summary
2) Problem Statement
3) Product Vision
4) Goals and Success Metrics
    Business goals (MVP)
    User goals (MVP)
    Success metrics (targets)
5) User Personas
6) Core Features
7) Feature Specifications
    7.1 Feature list (MoSCoW + effort)
    7.2 Detailed spec: "Severe rain → verified trigger → auto-claim → split payout"
8) User Flows (Mermaid)

9) System Architecture (MVP)
10) Data Requirements
    10.1 Data inputs
    10.2 Data outputs
11) AI/ML Requirements
    11.1 MVP approach
    11.2 Quality targets
12) API Specification (JSON examples)
    12.1 Create policy (high-level)
    12.2 Event detected → claim creation
13) Non-Functional Requirements
14) User Interface Requirements
    14.1 UX principles
    14.2 Core screens (MVP)
15) Security and Privacy
16) Milestones and Sequencing
17) Metrics and Event Tracking
    Tracking events
18) Risks and Mitigations
19) Out of Scope
20) Open Questions (resolve early)

## 1) Executive Summary

Resi-Gig is an AI-powered parametric insurance experience that provides **weekly income protection** to India's gig workers (starting with food delivery and quick-commerce riders) against income disruption caused by:

- Severe rain
- Bad air quality (AQI)
- Extreme heat
- Severe traffic combined with an event confirmation

Claims are **automated** (no paperwork). Payouts are **near-instant in the demo** (simulated split payout).

## 2) Problem Statement

Gig workers have volatile weekly earnings and limited financial buffers. Disruptions like monsoon rain and hazardous air quality can wipe out multiple shifts.

Traditional claims processes are slow and require documentation, leading to low trust and low utilization.

## 3) Product Vision

Enable gig workers to get trusted, fast income protection by converting disruption signals into:

- Transparent eligibility and decisioning
- Automatic claims creation and approvals
- Near-instant (simulated) payouts

**Vision statement:** “From disruption event to payout confirmation in minutes, without paperwork.”

## 4) Goals and Success Metrics

### Business goals (MVP)

- Demonstrate Guidewire **PolicyCenter** and **ClaimCenter** running an end-to-end automated flow.
- Build trust through transparent decisioning and audit-ready traces.
- Show fraud resistance and portfolio visibility for operators.

### User goals (MVP)

- Zero-touch onboarding completed in under 60 seconds.
- Personalized risk score and premium that feels fair.
- Automatic, paperwork-free claims and payout notifications.
- Simple dashboard for coverage and claim status.

### Success metrics (targets)

- Onboarding completion rate **> 95%** within **60 seconds** (p95 in demo).
- Claim accuracy **> 90%** against simulated ground truth.
- Notification-to-credit time **< 30 minutes** (simulated).
- Fraud rate **< 2%** flagged or reversed (demo assumption).

- Policy renewal rate **> 60%** (auto-renew opt-in).

## 5) User Personas

| Persona | Profile | Goals | Pain points | Device & behavior |
|---|---|---|---|---|
| Ravi the Rider (Primary) | Food delivery / quick-commerce rider, weekly net earnings ₹4,000–₹6,000 | Fast onboarding, predictable protection, instant clarity on coverage and claims | Earnings volatility, limited buffer, low trust in slow paperwork claims | Android, mobile-first, prefers 1-tap actions and clear status |
| Admin / Insurer Operator (Primary) | Operations user managing portfolio and claims performance | Monitor policies, claims, fraud signals, explainability | Needs auditability, low manual workload, and clear decision traces | Desktop-first dashboard, wants filters, queues, exports |
| Simulated Gig Platform Partner (Secondary) | Partner testing value and future integration potential | Aggregate insights, smoother onboarding + claims experience | Wants proof without committing to live integration in MVP | Occasional dashboard use, cares about trend summaries |

## 6) Core Features

- **Zero-touch onboarding (P0):** consent-driven GPS + mocked earnings and hours retrieval
- **Risk profiling + explainability (P0):** risk score (0–1), expected weekly loss, top factors ("why this price")
- **Quote + policy issuance (P0):** 7-day term with optional auto-renew; 3 coverage options
- **Parametric trigger detection + verification (P0):** rainfall, AQI, heat index, traffic+event confirmation
- **Auto-claim + decisioning (P0):** ClaimCenter creates claim; auto-approve above confidence threshold
- **Split payout simulation (P0):** 70% fast, 30% after secondary validation

- **Dashboards (P1):** Worker dashboard + Admin dashboard
- **Fraud + Trust Score (P1):** weekly refresh with drivers; influences discount/routing
- **What-if simulator (P1):** basic scenario simulation for users/admins

## 7) Feature Specifications

### 7.1 Feature list (MoSCoW + effort)

| Feature | Description | Priority (MoSCoW) | Effort (T-shirt) | Acceptance criteria |
|---|---|---|---|---|
| Consent + zero-input onboarding | Ask for GPS + simulated earnings/work-hours access; no typing | Must | M | ≥ 95% onboarding sessions complete in ≤ 60 seconds in demo |
| Risk scoring + explainability | Risk score (0–1), expected weekly loss (INR), top factors | Must | M | Quote renders within 3 seconds after data retrieval; shows top 3–5 factors |
| Quote + policy issuance | 3 coverage options; 7-day term; auto-renew optional | Must | M | Policy created in PolicyCenter and shown Active after confirmation |
| Parametric trigger detection | Detect/verify disruption events and geofence eligibility | Must | L | Claims created only for users inside geofence with active coverage |
| Auto-claim decisioning | ClaimCenter claim creation; auto-approve by confidence threshold | Must | M | Decision trace stored with sources + confidence for each claim |

| Feature | Description | Priority (MoSCoW) | Effort (T-shirt) | Acceptance criteria |
|---|---|---|---|---|
| Split payout simulation | 70% within 10–30 min; 30% after 24h validation (simulated) | Must | M | Payout status visible; no duplicate claims for same event+policy window |
| Worker dashboard | Coverage status, claims history, risk forecast, trust score | Should | M | Worker sees only their own policy and claim data |
| Admin dashboard | Active policies, approvals, fraud rate, heatmap, claim volumes | Should | L | Admin sees aggregate/ops views and review queues |
| Fraud + Trust Score | Isolation Forest + rules; weekly Trust Score (0–100) with drivers | Could | L | Fraud flags visible with reasons; Trust Score change includes drivers |
| What-if simulator | Scenario tests for risk and trigger rules | Could | M | User/admin can run at least one scenario and see expected outcomes |

### 7.2 Detailed spec: "Severe rain → verified trigger → auto-claim → split payout"

**Inputs**

- Coverage window (policy start/end) + rider GPS pings (mocked)
- Rain signal: rainfall > 40 mm/hour for 3+ hours
- Verification sources (mocked): Weather API, evidence service

**Processing**

- Detect event candidate, compute geofence and time window

- Verify event using sources and assign confidence
- Eligibility: rider must be inside affected zone during coverage window
- Create ClaimCenter claim with decision trace
- Auto-approve if confidence ≥ 85%, else route to “Needs Review”

**Outputs**

- Claim status visible to rider
- Payout simulation:
  - 70% credited within 10–30 minutes (simulated)
  - 30% after 24-hour validation pass (simulated)

## 8) User Flows (Mermaid)

```
flowchart TD
    A["Entry via QR / partner link<br>(Jutro mobile web)"]
--> B["Consent<br>GPS + simulated earnings"]
    B --> C["Auto-onboarding<br>Create Account + Draft Poli
cy"]
    C --> D["Risk score + quote<br>3 coverage options"]
    D --> E["Activate<br>Issue 7-day policy"]
    E --> F["Detect event<br>Rain/AQI/Heat/Traffic"]
    F --> G["Verify sources<br>Compute confidence"]
    G --> H{"Confidence ≥ 85%?"}
    H -->|Yes| I["Auto-approve claim<br>(ClaimCenter)"]
    H -->|No| J["Route to review queue"]
    I --> K["Payout simulation<br>70% now, 30% later"]
    J --> K
    K --> L["Worker dashboard<br>Status + history"]
```

## 9) System Architecture (MVP)

```
+-----------------------------+
| Frontend (MVP)              |
| Jutro + React mobile-first  |
+--------------+--------------+
               |
```

```
                 v
+-----------------------------------------------------+
| Backend Orchestrator (Node.js / Flask)              |
| - Mock data retrieval (earnings, hours, gps)        |
| - Risk scoring + explainability                     |
| - Trigger detection + verification                  |
| - PolicyCenter + ClaimCenter orchestration          |
| - Payout simulation                                 |
+------------------+------------------+-------------+
                   |                  |
                   v                  v
      +--------------------+   +----------------------+
      | External signals   |   | Guidewire Cloud      |
      | Weather/AQI/Traffic|   | PolicyCenter         |
      | OSINT (mocked)     |   | ClaimCenter          |
      +--------------------+   +----------------------+
```

## 10) Data Requirements

### 10.1 Data inputs

- Consent flags (GPS, mock earnings/hours)
- Earnings log (mock)
- Work schedule (mock)
- Weather / AQI / traffic signals (mocked APIs)
- Evidence snippets (mocked OSINT/news)

### 10.2 Data outputs

- Risk profile: risk score, expected weekly loss, top factors
- Policy: term window, coverage selection, premium, trust discount
- Trigger event: type, severity, geofence, sources, confidence
- Claim: eligibility result, decision trace, status
- Payout: split payouts, timestamps, status
- Trust Score: weekly value + drivers

## 11) AI/ML Requirements

### 11.1 MVP approach

- Risk scoring: Random Forest / XGBoost (may be simplified for MVP)
- Fraud detection: Isolation Forest + rules
- Explainability: top contributing factors for pricing and traces for claims

### 11.2 Quality targets

- Quote and risk explanation visible within 3 seconds after data retrieval
- Claim decision trace captured for every claim

## 12) API Specification (JSON examples)

### 12.1 Create policy (high-level)

**POST /policies**

```
{
  "rider_id": "r_123",
  "coverage_option": "full_week",
  "term_days": 7,
  "auto_renew": true
}
```

### 12.2 Event detected → claim creation

**POST /events/detect**

```
{
  "event_type": "rain",
  "threshold": { "mm_per_hr": 40, "hours": 3 },
  "geofence": "zone_45",
  "observed_window": "2026-04-01T08:00:00Z/2026-04-01T11:0
0:00Z"
}
```

**Response**

```
{
  "event_id": "evt_456",
  "confidence": 0.9,
  "claims_created": 128,
  "routed_to_review": 7
}
```

## 13) Non-Functional Requirements

- **Performance:** onboarding end-to-end under 60 seconds for p95 in demo
- **Reliability:** 99% uptime for core demo flows
- **Security/privacy:** encrypt GPS and earnings at rest; hard expiry for session data; no permanent PII retention in MVP
- **Explainability:** every claim decision stores a decision trace (inputs, rules, sources, confidence)

## 14) User Interface Requirements

### 14.1 UX principles

- Mobile-optimized, minimal text, large CTAs
- Always show status clearly: Coverage Active, Claim Created, Approved, Payout Pending/Complete
- “Why this price” and “Why this claim decision” must be accessible in 1–2 taps

### 14.2 Core screens (MVP)

- Entry + consent
- Quote selection (3 options)
- Coverage dashboard (policy status, forecast)
- Claim timeline (event → verification → decision → payout)
- Admin dashboard (portfolio, queues, fraud flags)

## 15) Security and Privacy

- No permanent PII retention in MVP (demo accounts)
- Session data expires automatically
- Encrypt sensitive fields at rest (GPS, earnings)
- Audit-friendly decision traces for every claim

## 16) Milestones and Sequencing

1. **Phase 1 (Week 1–2): Core MVP**
    - Onboarding flow and mock APIs
    - Risk scoring and explainability
    - Policy issuance in PolicyCenter
    - One trigger end-to-end (rainfall recommended)
2. **Phase 2 (Week 3): Automation and payouts**
    - Multi-source verification engine
    - ClaimCenter workflows
    - Split payout simulation
    - Initial fraud signals
3. **Phase 3 (Week 4): Polish and demo**
    - Worker and Admin dashboards
    - Basic what-if simulator
    - Data seeding and demo script

## 17) Metrics and Event Tracking

### Tracking events

- onboarding_started
- onboarding_completed
- policy_selected
- policy_activated

- claim_triggered
- claim_verified
- payout_initiated
- payout_completed
- trust_score_updated
- dashboard_viewed

## 18) Risks and Mitigations

| Risk | Impact | Likelihood | Mitigation |
|---|---|---|---|
| False positives on triggers | Unfair claims; cost and trust issues | Medium | Multi-source verification; confidence threshold; review queue |
| Missing geo-proof (eligibility disputes) | Disputes; poor trust | Medium | Clear geofence logic; store GPS evidence + timestamps in trace |
| Latency exceeds demo targets | Weak value proposition | Medium | Cache signals; precompute; async processing for non-blocking steps |
| Fraud simulation not credible | Operator skepticism | Low | Seed realistic scenarios; show flags with reasons and outcomes |

## 19) Out of Scope

- Real UPI money movement, KYC, regulated compliance flows
- Native mobile app
- Live integrations with actual gig platforms
- Full multilingual support beyond essential English and Hindi

## 20) Open Questions (resolve early)

- MVP launch city and geofence granularity (ward, neighborhood, pin-code)
- Coverage limit per week and per trigger type in the demo
- Ground truth method for simulated claim correctness

- How explainability is shown in the UI (top factors, evidence snippets)
- Admin role model (permissions and audit needs)